Contender 20: W-Rank Hyperreflection in BusyLean

A second-order, well-founded-tree closure of diagonal transformer lifting

Research proposal prepared for Vladimir Reshetnikov

October 13, 2026

Abstract

BusyLean's current Contender 19 combines two independent sources of growth: a large-Veblen-length ladder of normalizing universes and a finite hierarchy of *transformer lifts*. Its lift rank is a natural number, albeit the astronomical value family18$(n) + 1$. This note proposes Contender 20, which raises the amplifier one type level. A *hyperlift* maps transformer lifts to transformer lifts. We define a structural recursion that iterates a hyperlift along an arbitrary countably branching well-founded tree. At each outer node, the recursively generated child lifts are spliced into a single lift, after which one diagonal rank step is applied. This is a W-tree analogue of transfinite iteration with a genuine limit constructor, rather than another finite self-composition.

The central technical point is strict growth without monotonicity. Reflection is not monotone in its oracle: a reflected program may inspect an oracle value and branch. Consequently, none of the comparison proofs may replace an oracle by a pointwise larger one. We instead maintain two explicit invariants: a lift preserves progressive transformers, and it strictly exceeds its input transformer at every visible coordinate. We prove that splicing preserves both invariants, one rank step preserves both, W-tree iteration preserves both, and every nonempty stage—including a leaf stage—strictly exceeds its seed lift. The node proof follows child zero and then coordinate zero of the splice; it uses no extensional monotonicity principle.

At input n, the finite seed lift already has rank family19$(n)+1$. The outer rank is an autonomous hyperclosure above a schedule that is itself an autonomous hyperclosure above Contender 19's schedule. Applying the resulting W-rank lift to the large-Veblen reflector and the constant oracle family family19 yields $F_{20}(n)$, and contender$_{20} = F_{20}(F_{20}(42))$. The supplied Lean development proves pointwise family19$(n) < F_{20}(n)$, hence contender$_{19} <$ contender$_{20}$, and also bounds every large-Veblen-ladder program in the existing depth-n enumeration.

The package is designed for BusyLean's Lean 4.34.0 toolchain and includes the three Lean modules, focused audit and smoke files, an integration patch, and static checks. The present execution environment did not contain Lean, so the sources have not been kernel-checked here; this verification boundary is stated explicitly throughout the report.

Contents

1 Result at a glance **4**

2 Baseline: what Contender 19 already does **4**

3 Design criteria and rejected easy moves **5**

4 Four type levels of reflection data **6**

5 The invariants: progress and strict occurrence **6**

6 Splicing lifts: the limit mechanism **7**

7 One diagonal step at the hyperlift level **8**

8 W-tree iteration of hyperlifts **8**

9 The generic W-rank diagonal lift **10**

10 Choosing the two Contender-20 trees **10**

11 The finite seed lift and the final family **11**

12 Formal comparison with Contender 19 **11**
12.1 The previous family is below raw reflection . . . 11
12.2 Raw reflection is below the finite seed lift . . . 11
12.3 The finite seed lift is below the W-rank lift . . . 12
12.4 Pointwise and closed comparisons . . . 12

13 Bounding the large-Veblen programs **12**

14 Computability and termination **13**
14.1 Structural definitions . . . 13
14.2 Countable branching is still data . . . 13
14.3 Why evaluation should not be attempted . . . 13

15 How much larger is it? **13**
15.1 The formal answer . . . 13
15.2 The heuristic answer . . . 14

16 Why the proposal is not a disguised finite rank **14**

17 Audit and engineering considerations **15**
17.1 Dependency discipline . . . 15
17.2 Axiom audit . . . 15
17.3 Static audit already performed . . . 15
17.4 Expected build cost . . . 15
17.5 Repository integration . . . 16

18 Verification boundary and risk register **16**

19 Further research program **17**
19.1 Immediate formal tasks . . . 17
19.2 Semantics and ordinal analysis . . . 17
19.3 Higher type levels . . . 17
19.4 Reflecting the amplifier language . . . 17
19.5 Collapsing-function universe ladders . . . 18
19.6 Minimality and compression . . . 18

20 Conclusion **18**

A Complete generic Lean module **18**

B Complete contender definition **21**

C Complete comparison module **23**

D Reproducibility checklist **24**

1 Result at a glance

The proposal adds one generic module and two contender-specific modules:

File	Role
`TransfiniteHyperReflection.lean`	Hyperlifts, lift splicing, W-tree iteration, preservation and strictness theorems.
`Contender20.lean`	The two autonomous schedules, the finite seed lift, the W-rank lift, F_{20}, and `contender_20`.
`Comparison20.lean`	The pointwise comparison chain, `contender_19_lt_contender_20`, and the program-value bound.

The headline definition is short:

```
def family19 (n : Nat) : Nat := F19 (F19 n)

def seedLift20 (n : Nat) : TransformerLift :=
  diagonalLift (schedule20 n) (family19 n + 1)

def lift20 (n : Nat) : TransformerLift :=
  treeDiagonalLift (schedule20 n) (rankTree20 n) (seedLift20 n)

def F20 (n : Nat) : Nat :=
  lift20 n lTransformer seedFamily20 n n

def contender_20 : Nat := F20 (F20 42)
```

The proof is deliberately factored into three strict inequalities:

$$\underbrace{\mathrm{family19}(n)}_{F_{19}(F_{19}(n))} < \underbrace{L(\mathrm{seed}_{20})}_{\text{one large-Veblen reflection}} < \underbrace{D_n(L)(\mathrm{seed}_{20})}_{\text{finite-rank seed lift}} < \underbrace{W_n(D_n)(L)(\mathrm{seed}_{20})}_{F_{20}(n)}. \tag{1}$$

Here L is the existing large-Veblen reflector, D_n is a diagonal lift of rank $\mathrm{family19}(n)+1$, and W_n is the new W-tree rank closure. Each arrow is witnessed by an existing or newly proved strict-inflation property; no appeal is made to an informal ordinal estimate.

Formal status. The package gives complete Lean terms for the definitions and proofs. Static checks confirm balanced delimiters, line lengths below 81 in the three main modules, and absence of the tokens used for nonconstructive or incomplete shortcuts. Because no Lean executable was available in the artifact runtime, "the Lean development proves" in this note means that the supplied source contains the stated proof term and is intended to pass the repository's kernel build; an actual build receipt remains the first integration task.

2 Baseline: what Contender 19 already does

BusyLean is a Lean 4 continuation of the constructive biggest-number contest originating in Roux's Coq repository and Aaronson's larger-number essay [1, 2]. A contender is not merely a description. It is a closed `Nat`, computable in principle, together with a formal theorem that it is larger than the incumbent. The repository additionally audits axioms, dependency closure, and clean-build time.

Contender 19 has the form

$$F_{19}(n) = D_{S_{19}(n),\, r_{19}(n)}(L)(\lambda i.\,\mathrm{family18})(n,n), \qquad \mathrm{contender}_{19} = F_{19}(F_{19}(42)),$$

where:

- $L = \texttt{lTransformer}$ reflects over the level-n language whose atoms are normal large-Veblen terms;
- $S_{19}(n)$ is an autonomous W-tree schedule above the earlier index;
- $D_{S,k} = \texttt{diagonalLift S k}$ is a natural-number hierarchy of transformer lifts;
- $r_{19}(n) = \text{family18}(n) + 1$; and
- $\text{family18}(n) = F_{18}(F_{18}(n))$.

The language side is already formidable. Its atoms are Schütte-style Klammersymbole: Veblen argument arrays whose positions are themselves terms. The repository proves well-foundedness of the normal forms and then obtains a normalizing universe ladder from the generic theorem `FL.wfUniv`. In particular, a new contender cannot honestly advertise a major advance merely by replacing 42 with 43 or adding one more self-composition.

The amplifier side, however, has a clean boundary. `diagonalLift` is indexed by $k \in \mathbb{N}$:

$$D_{S,0}(T) = \texttt{liftTransformer}(T),$$
$$D_{S,k+1}(T) = \texttt{stageLift}(D_{S,k})(D_{S,k}(T))(S).$$

Even when k is incomprehensibly large, the rank object is still a natural number. Contender 20 targets precisely this boundary.

3 Design criteria and rejected easy moves

A satisfactory successor should meet five criteria.

1. **Closed and constructive.** Every definition must reduce by structural recursion or already certified normalization; no unbounded search for a witness, semantic truth predicate, or Busy Beaver is permitted.
2. **A structural advance.** The definition should add a new closure principle, not only a larger numeral in an existing parameter.
3. **A direct formal comparison.** The previous family should occur as a computable seed, while the proof should expose strict occurrences rather than rely on unproved growth folklore.
4. **Compatibility with nonmonotone reflection.** Reflected programs can branch on oracle values, so pointwise oracle monotonicity is unavailable.
5. **Small trusted surface.** The new proof should be generic and sit above the existing normalization theorems rather than duplicate the large-Veblen development.

Several tempting moves were rejected.

Only seed Contender 19 into Contender 19's amplifier. Replacing `family18` by `family19` and retaining exactly the old finite-rank construction would certainly give a larger number and a short proof. It would also be a routine generational step. The present proposal uses that stronger seed *and* adds an independent W-rank closure.

Add more finite self-compositions. Definitions such as $F_{19}^{F_{19}(42)}(42)$ are computable and enormous, but they do not exploit BusyLean's generic reflection architecture. They also obscure the conceptual comparison between language strength and amplifier strength.

Jump immediately to collapsing-function atoms. A Bachmann–Howard or stronger ladder is a natural language-side project, but raw collapsing-function syntax is not automatically well-founded under the intended comparison. Canonical closure side conditions are essential, and a credible formalization is substantially larger than the present overlay. It is a valuable future direction, not a prudent source of a quick theorem.

Assume monotonicity. This is simply invalid. If a reflected term reads an oracle value and branches, a larger oracle can select a smaller branch. Existing BusyLean proofs avoid this trap, and the new second-order proofs preserve that discipline.

4 Four type levels of reflection data

The construction is easiest to understand by separating four function spaces.

Definition 4.1 (Families)**.**

$$\mathsf{Fam} := \mathbb{N} \to \mathbb{N} \to \mathbb{N}.$$

A family s is read as a countable collection of unary functions: $s(i, n)$ is coordinate i at numerical input n.

Definition 4.2 (Transformers)**.**

$$\mathsf{Tr} := \mathsf{Fam} \to \mathsf{Fam}.$$

A transformer maps an oracle family to a new family. The repository's large-Veblen reflection transformer is one example.

Definition 4.3 (Transformer lifts)**.**

$$\mathsf{Lift} := \mathsf{Tr} \to \mathsf{Tr}.$$

A transformer lift amplifies a transformer. Existing examples are `liftTransformer`, `stageLift L`, and every `diagonalLift schedule k`.

Definition 4.4 (Hyperlifts)**.**

$$\mathsf{HLift} := \mathsf{Lift} \to \mathsf{Lift}.$$

This is the new level. A hyperlift amplifies the method by which transformers are amplified.

The Lean declarations mirror this hierarchy:

```
abbrev Transformer := Family → Family
abbrev TransformerLift := Transformer → Transformer
abbrev HyperLift := TransformerLift → TransformerLift
```

A type-level rise of this kind has precedent inside BusyLean. Contender 6 iterated ordinary reflection over families. Contender 12 lifted reflection to transformers. Contender 15 introduced a finite hierarchy of transformer lifts. Contender 20 moves from a natural-number hierarchy in Lift to well-founded-tree recursion in HLift.

5 The invariants: progress and strict occurrence

The first-order invariant already present in BusyLean is progressivity.

Definition 5.1 (Progressive transformer)**.** A transformer T is progressive when

$$\mathsf{Prog}(T) :\Longleftrightarrow \forall s, i, o, n\ \big(i \le n \Rightarrow s(i,n) < T(s)(o,n)\big).$$

Every visible seed coordinate is strictly below every output coordinate at the same numerical input.

For a lift $L \in \mathsf{Lift}$, two properties matter.

Definition 5.2 (Preservation and strict inflation)**.**

$$\begin{aligned}\mathsf{Pres}(L) &:\Longleftrightarrow \forall T\ (\mathsf{Prog}(T) \Rightarrow \mathsf{Prog}(L(T))),\\ \mathsf{Strict}(L) &:\Longleftrightarrow \forall T\ \Big(\mathsf{Prog}(T) \Rightarrow \forall s, i, o, n\ (i \le n \Rightarrow T(s)(i,n) < L(T)(s)(o,n))\Big).\end{aligned}$$

Set $\mathsf{Good}(L) := \mathsf{Pres}(L) \wedge \mathsf{Strict}(L)$.

In Lean these are the existing predicates `PreservesProgressive` and `StrictlyInflationary`, paired by the new abbreviation:

```
abbrev GoodLift (lift : TransformerLift) : Prop :=
  PreservesProgressive lift ∧ StrictlyInflationary lift
```

At the hyperlift level we require preservation of goodness and a second-order strictness statement.

Definition 5.3 (Goodness-preserving and second-order strict)**.** For $H \in \mathsf{HLift}$,

$$\begin{aligned}\mathsf{PresGood}(H) &:\Longleftrightarrow \forall L\ (\mathsf{Good}(L) \Rightarrow \mathsf{Good}(H(L))),\\ \mathsf{Strict}_2(H) &:\Longleftrightarrow \forall L\ \big[\mathsf{Good}(L) \Rightarrow\\ &\qquad \forall T\ \big[\mathsf{Prog}(T) \Rightarrow\\ &\qquad\quad \forall s, i, o, n\ (i \le n \Rightarrow L(T)(s)(i,n) < H(L)(T)(s)(o,n))\big]\big].\end{aligned}$$

These predicates are intentionally intensional: they identify a concrete copy of the old object inside the new one. That is why they remain useful when no monotonicity theorem exists.

6 Splicing lifts: the limit mechanism

The repository already has `spliceTransformers`. Given transformers $T_0, T_1, \ldots$, the splice selects T_i by output coordinate i, then reads its canonical coordinate zero. We lift this operation one type level.

Definition 6.1 (Lift splice)**.** For $L_i \in \mathsf{Lift}$, define

$$\mathsf{spliceLift}(L_0, L_1, \ldots)(T) := \mathsf{spliceTransformer}(L_0(T), L_1(T), \ldots).$$

Equivalently,

$$\mathsf{spliceLift}(\vec{L})(T)(s)(i,n) = L_i(T)(s)(0,n).$$

```
def spliceLifts (lifts : Nat → TransformerLift) : TransformerLift :=
  fun base => spliceTransformers (fun i => lifts i base)
```

Proposition 6.2 (Splicing preserves goodness)**.** *If every L_i is good, then* $\mathsf{spliceLift}(\vec{L})$ *is good.*

Proof. Let T be progressive. Each $L_i(T)$ is progressive, so the existing first-order splice theorem makes their splice progressive.

For strictness, fix seed s, input coordinate j, output coordinate i, and $j \le n$. Strictness of L_i, with canonical output coordinate zero, gives

$$T(s)(j,n) < L_i(T)(s)(0,n) = \mathsf{spliceLift}(\vec{L})(T)(s)(i,n).$$

No comparison between two different oracle families occurs. □

The splice is the correct analogue of a limit-stage join. It does not collapse all children to a numerical maximum. Every child remains addressable, which allows later transformers to select branches dynamically through the family interface.

7 One diagonal step at the hyperlift level

Fix an inner schedule $S \in \mathsf{WTree}(\mathbb{N})$. The new hyperlift Step_S sends a lift L to

$$\mathsf{Step}_S(L)(T) := \texttt{stageLift}(L, L(T), S).$$

```
def rankStep (schedule : WTree Nat) : HyperLift :=
  fun lift base => stageLift lift (lift base) schedule
```

This is exactly the successor functional implicit in `diagonalLift`: the input lift appears twice, once to pre-amplify the base transformer and once as the local lifting operation used at every node of the schedule.

Proposition 7.1 (Rank step preserves goodness)**.** *If L is good, then $\mathsf{Step}_S(L)$ is good.*

Proof. Let T be progressive. Preservation for L gives progressivity of $L(T)$. The existing theorem `stageLift_progressive` now shows that $\texttt{stageLift}(L, L(T), S)$ is progressive.

For strictness, use two explicit strict inclusions:

$$T(s)(i,n) < L(T)(s)(i,n) < \texttt{stageLift}(L, L(T), S)(s)(o,n).$$

The first is strictness of L; the second is `base_lt_stageLift`, applied with base transformer $L(T)$. □

Proposition 7.2 (Second-order strictness)**.** *If L is good and T is progressive, then*

$$L(T)(s)(i,n) < \mathsf{Step}_S(L)(T)(s)(o,n) \qquad (i \le n).$$

Thus $\mathsf{Strict}_2(\mathsf{Step}_S)$ holds.

Proof. This is a direct application of `base_lt_stageLift` with base transformer $L(T)$. Unlike Proposition 7.1, no preliminary step from T to $L(T)$ is needed. □

8 W-tree iteration of hyperlifts

Let $H \in \mathsf{HLift}$, let $L_0 \in \mathsf{Lift}$, and let $A \in \mathsf{WTree}(\mathbb{N})$. Define $\mathsf{Run}(H, L_0, A) \in \mathsf{Lift}$ by structural recursion:

$$\begin{aligned} \mathsf{Run}(H, L_0, \mathsf{leaf}) &= H(L_0), \\ \mathsf{Run}(H, L_0, \mathsf{node}(f)) &= H\left(\mathsf{spliceLift}(i \mapsto \mathsf{Run}(H, L_0, f(i)))\right). \end{aligned}$$

```
def runHyperLift (op : HyperLift) (seed : TransformerLift) :
    WTree Nat → TransformerLift
  | .leaf => op seed
  | .node f =>
      op (spliceLifts (fun i => runHyperLift op seed (f i)))
```

Because `WTree` is inductive, every branch is well-founded even though a node has countably many children and their heights may be unbounded. Hence the outer rank can represent genuine limit shapes such as a root whose i-th child has finite height i. The definition remains ordinary structural recursion in Lean.

Theorem 8.1 (Goodness of W-tree iteration)**.** *If H preserves good lifts and L_0 is good, then $\mathsf{Run}(H, L_0, A)$ is good for every well-founded tree A.*

Proof. By structural induction on A. At a leaf, apply preservation of goodness to L_0. At a node, the induction hypothesis makes every child lift good; Proposition 6.2 makes their splice good; preservation of goodness for H finishes the step. □

The next theorem is the key comparison result.

Theorem 8.2 (The W-tree stage strictly exceeds its seed)**.** *Assume H preserves goodness and is second-order strictly inflationary. If L_0 is good and T is progressive, then for every A, seed family s, and $i \le n$,*

$$L_0(T)(s)(i,n) < \mathsf{Run}(H, L_0, A)(T)(s)(o,n).$$

Proof. Again use structural induction on A.

At a leaf, the result is exactly $\mathsf{Strict}_2(H)$.

At a node $\mathsf{node}(f)$, first follow child zero. The induction hypothesis gives

$$L_0(T)(s)(i,n) < \mathsf{Run}(H, L_0, f(0))(T)(s)(0,n).$$

By definition of lift splicing, the right-hand side is coordinate zero of the splice of all child lifts. Theorem 8.1 and Proposition 6.2 show that this splice is good. Apply $\mathsf{Strict}_2(H)$ to the splice, from its coordinate zero to arbitrary output coordinate o:

$$\mathsf{Run}(H, L_0, f(0))(T)(s)(0,n) < H(\mathsf{spliceLift}(\ldots))(T)(s)(o,n).$$

The last term is definitionally $\mathsf{Run}(H, L_0, \mathsf{node}(f))(T)(s)(o,n)$. Transitivity completes the proof. □

Remark 8.3 (Why child zero is enough)*.* The theorem does not attempt to compare every child simultaneously or take a maximum over child values. It needs one explicit embedded predecessor. Child zero supplies that predecessor, while the splice preserves access to all other children for subsequent computations.

Remark 8.4 (No hidden monotonicity)*.* The proof never changes the oracle supplied to the same reflected program. Every inequality follows a syntactically present old lift or child. This is the second-order analogue of the proof style already used by BusyLean's `transformer_lt_lift_at` and `base_lt_stageLift`.

9 The generic W-rank diagonal lift

We specialize the preceding recursion to the diagonal rank step:

$$\mathsf{treeDiagonalLift}(S, A, L_0) := \mathsf{Run}(\mathsf{Step}_S, L_0, A).$$

```
def treeDiagonalLift (schedule rankTree : WTree Nat)
    (seed : TransformerLift) : TransformerLift :=
  runHyperLift (rankStep schedule) seed rankTree
```

Propositions 7.1 and 7.2, together with Theorems 8.1 and 8.2, immediately yield:

Corollary 9.1 (Generic W-rank facts)**.** *If L_0 is good, then* $\mathsf{treeDiagonalLift}(S, A, L_0)$ *is good and strictly exceeds L_0 at every visible coordinate over every progressive base transformer.*

This factorization is important for auditability. The contender-specific file does not prove a bespoke inequality through an enormous definition. It only instantiates a small, reusable theorem about arbitrary schedules, rank trees, seed lifts, base transformers, and oracle families.

10 Choosing the two Contender-20 trees

Let

$$\mathrm{family19}(n) := F_{19}(F_{19}(n)).$$

As in earlier BusyLean generations, this is a function of the input, not a reference to the closed constant `contender_19`. Consequently the closure audit can require that `contender_20` contain none of the earlier closed contender constants.

The repository's `Autonomous` syntax builds transparent well-founded trees with addition, multiplication, omega powers, derivative-like fixed-point closures, a common-critical closure, and higher derivatives. We use it twice.

Construction 10.1 (Inner schedule)**.**

$$S_{20}(n) := \texttt{Autonomous.realize}\big(\texttt{Autonomous.hyper}(\mathrm{family19}(n) + 42, \texttt{Autonomous.encode}(S_{19}(n)))\big).$$

Construction 10.2 (Outer rank tree)**.**

$$R_{20}(n) := \texttt{Autonomous.realize}\big(\texttt{Autonomous.hyper}(\mathrm{family19}(n) + 42, \texttt{Autonomous.encode}(S_{20}(n)))\big).$$

The first closure enlarges the schedule used inside each rank step. The second closure is not another inner schedule: it indexes the recursion over lifts itself. This separation prevents the outer mechanism from collapsing into "one more finite rank" by notation alone.

The concrete Lean code is:

```
def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42)
      (Autonomous.encode (schedule19 n)))

def rankTree20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42)
      (Autonomous.encode (schedule20 n)))
```

Caution 10.3 (Syntax trees, not asserted ordinal equalities). BusyLean's `Autonomous` module explicitly treats these operations as transparent tree syntax. The present proposal likewise does not identify `hyper k a` with a particular quotient ordinal. The only formal facts used are structural well-foundedness of the generated W-trees and the generic strict-growth theorems.

11 The finite seed lift and the final family

The outer W-rank iteration starts from a seed that already exceeds the entire Contender-19 family pointwise through a large finite rank:

$$D_{20}(n) := \texttt{diagonalLift}(S_{20}(n), \mathrm{family19}(n) + 1).$$

This is `seedLift20 n`. Existing BusyLean theorems give both components of $\mathsf{Good}(D_{20}(n))$.

Define

$$W_{20}(n) := \mathsf{treeDiagonalLift}(S_{20}(n), R_{20}(n), D_{20}(n)).$$

Let $s_{20}(i, n) = \mathrm{family19}(n)$ be the constant seed family. Finally,

$$F_{20}(n) := W_{20}(n)(L)(s_{20})(n, n), \qquad \mathrm{contender}_{20} := F_{20}(F_{20}(42)),$$

where $L = \texttt{lTransformer}$.

The full contender file has only six substantive definitions after the imports. There is no new object-language interpreter, type checker, or normalization argument. All language-theoretic strength is inherited from Contender 19; all new work lies in the generic amplifier.

12 Formal comparison with Contender 19

We now unpack the chain (1).

12.1 The previous family is below raw reflection

Progressivity of the large-Veblen transformer states

$$s(i, n) < L(s)(o, n) \qquad (i \le n).$$

Taking $s = s_{20}$, $i = o = 0$, and observing that $s_{20}(0, n) = \mathrm{family19}(n)$, we obtain

$$\mathrm{family19}(n) < L(s_{20})(0, n).$$

This is `family19_lt_raw20`.

12.2 Raw reflection is below the finite seed lift

The existing theorem `diagonalLift_strictlyInflationary` gives

$$L(s_{20})(0, n) < D_{20}(n)(L)(s_{20})(0, n).$$

This is `raw20_lt_seedLift20`.

12.3 The finite seed lift is below the W-rank lift

Corollary 9.1, instantiated with schedule $S_{20}(n)$, rank tree $R_{20}(n)$, seed lift $D_{20}(n)$, and base transformer L, gives

$$D_{20}(n)(L)(s_{20})(0,n) < W_{20}(n)(L)(s_{20})(n,n) = F_{20}(n).$$

This is `seedLift20_lt_F20`. Notice that this theorem isolates the new structural gain: even after the finite seed rank has been chosen using family19(n), the outer W-tree stage is formally and strictly larger.

12.4 Pointwise and closed comparisons

By transitivity,

Theorem 12.1 (Pointwise dominance)**.** *For every n,*

$$\text{family19}(n) < F_{20}(n).$$

The existing input-growth theorem for F_{19} gives

$$n < F_{19}(n) < F_{19}(F_{19}(n)) = \text{family19}(n) < F_{20}(n),$$

so $n < F_{20}(n)$ for every n. At $n = 42$,

$$\text{contender}_{19} = \text{family19}(42) < F_{20}(42) < F_{20}(F_{20}(42)) = \text{contender}_{20}.$$

Thus the proposed module states:

```
theorem contender_19_lt_contender_20 :
    contender_19 < contender_20 :=
  Nat.lt_trans (family19_lt_F20 42)
    (input_lt_F20 (F20 42))
```

The same file records the inherited corollary $\text{contender}_{18} < \text{contender}_{20}$.

13 Bounding the large-Veblen programs

A contender should not merely dominate the previous closed numeral; it should retain a semantic explanation. Contender 19 defines `FL.lveblenLadderUpTo n`, the finite enumeration of large-Veblen-ladder programs of depth at most n, and `FL.lvalue` for their normalized natural-number values relative to a packed oracle family.

The existing theorem says that every enumerated value is below one raw large-Veblen reflection:

$$\texttt{FL.lvalue}(\texttt{packFamily}(s), n, t) < \texttt{reflectLFamily}(s, n).$$

Combining it with raw reflection below $F_{20}(n)$ yields:

Theorem 13.1 (Program bound)**.** *If $t \in$ `FL.lveblenLadderUpTo n`, then*

$$\textit{FL.lvalue}(\textit{packFamily}(s_{20}), n, t) < F_{20}(n).$$

This theorem, named `lvalue_lt_F20`, ties the new amplifier back to the strongest object language in the repository. It also shows why reusing the large-Veblen reflector is preferable to an arbitrary arithmetic base: the number continues to dominate an explicit, certified class of programs.

14 Computability and termination

The number is computationally hopeless but constructive in the contest's sense.

14.1 Structural definitions

The new definitions recurse only on values whose inductive structure is visible:

- `runHyperLift` recurses on `WTree Nat`;
- `Autonomous.hyper` recurses on a natural number and builds `OTree` syntax;
- `Autonomous.realize` and `encode` recurse on tree syntax;
- `diagonalLift` recurses on a natural-number rank; and
- the inherited normalizer computes only from an explicit strong- normalization certificate.

No new semantic termination axiom is introduced. In particular, the outer rank tree is not traversed by a general while loop or a minimization operator; it is consumed by the recursor of an inductive W-type.

14.2 Countable branching is still data

A node of `WTree Nat` stores a total function $\mathbb{N} \to \mathsf{WTree}(\mathbb{N})$. Such a function can describe infinitely many children, but it is itself a finite Lean term computing any requested child. The inductive type rules out infinite descending branches. This is exactly the representation already used by the repository's forest reflection and autonomous schedules.

14.3 Why evaluation should not be attempted

The fact that a term denotes a natural number and is computable in principle does not imply that reduction is remotely feasible. Merely constructing the rank family19(42) + 1 is beyond physical possibility, before any large-Veblen program normalization or outer W-rank traversal occurs. The proper artifact is the definition and strict comparison theorem, not decimal or binary expansion.

15 How much larger is it?

There are two distinct answers.

15.1 The formal answer

The pointwise theorem

$$\forall n \quad \text{family19}(n) < F_{20}(n)$$

is exact. The proof also factors through two additional strict stages after raw reflection. Therefore the new family is not merely larger at the final input; it uniformly dominates the self-composed Contender-19 family.

Moreover, `seedLift20_lt_F20` proves that the new W-rank stage adds a strict amount even after the finite diagonal rank family19$(n) + 1$ has been applied. This isolates a formal advantage of the new mechanism over the strongest obvious same-type seed.

15.2 The heuristic answer

Let $\Phi_S(L) = \mathsf{Step}_S(L)$. Existing `diagonalLift S k` is finite iteration of a closely related successor operation beginning with autonomous lifting. In contrast, `runHyperLift` supports a W-tree recursion: children are recursively evaluated, retained separately through a splice, and then passed through Φ_S. A linear rank tree behaves like a long iterate of Φ_S; a limit-shaped node behaves like a join of a countable family of prior iterates followed by another step.

This makes `treeDiagonalLift` a natural intensional analogue of transfinite iteration of Φ_S. The rank tree $R_{20}(n)$ is itself an autonomous hyperclosure above $S_{20}(n)$, so the outer closure is far more than a fixed finite number of successor steps.

Caution 15.1 (No exact ordinal calibration is claimed). The operations manipulate program transformers that can inspect oracles. Their extensional behavior is nonmonotone, and the autonomous trees are syntax rather than quotient ordinals. Assigning a precise fast-growing-hierarchy index would require a semantics of lift expressions and a simulation theorem. The report therefore uses "W-rank" and "transfinite-shaped" descriptively, not as a proved equality with a named ordinal.

A conservative qualitative summary is:

Construction	Rank object	Limit behavior
Contender 15–19 amplifier	natural number	none at rank level
Contender 20 seed	natural number family$19(n) + 1$	none at rank level
Contender 20 outer stage	well-founded countably branching tree	splice of child lifts

16 Why the proposal is not a disguised finite rank

One might worry that every concrete Lean term is finite syntax, so the outer W-tree could secretly amount to one very large natural rank. This objection confuses syntax size with inductive rank. The term

$$\mathsf{node}(i \mapsto \mathsf{chain}(i))$$

is finite syntax but has children of every finite height. There is no single finite branch-height bound. More elaborate definable child functions produce far larger well-founded ranks.

The outer recursion also retains child lifts separately. Replacing a node by "iterate the same lift m times" would discard this addressability. In `runHyperLift`, later rank steps receive a splice and can select different child lifts by output coordinate. Thus even a numerical upper bound on the height would not by itself establish extensional equivalence to `diagonalLift S k`.

A fully formal separation theorem is left open. Useful targets are:

1. prove that chain-shaped rank trees simulate specified finite hyperlift iterates;
2. prove that a canonical omega-shaped rank tree dominates every fixed finite iterate on a family of test transformers; and
3. derive a non-representability result for W-rank lifts analogous to the repository's earlier language non-representability results.

The present contender does not need this separation theorem for strict comparison, because Theorem 8.2 already proves that the outer stage strictly exceeds its chosen finite seed lift.

17 Audit and engineering considerations

17.1 Dependency discipline

The definition uses `F19` and `schedule19` but never the closed constant `contender_19`. This follows the repository's established pattern: each generation depends on the previous *family*, not the previous final constant. The focused audit traverses the environment closure of `contender_20` and rejects dependencies on `contender_5`,...,`contender_19`.

17.2 Axiom audit

The package asks Lean to print axioms for:

- `contender_20`;
- `contender_19_lt_contender_20`;
- `family19_lt_F20`;
- `lvalue_lt_F20`;
- `seed_lt_runHyperLift`; and
- `treeDiagonalLift_good`.

The intended result is the repository's permitted set, at most `propext` and `Quot.sound`, with no `Classical.choice`. The new proofs themselves use only induction, conjunction, definitional unfolding, and transitivity of $<$.

17.3 Static audit already performed

The included `scripts/static_audit.py` strips nested Lean comments and checks the three main modules for:

- balanced parentheses, brackets, and braces;
- forbidden shortcut tokens;
- SHA-256 digests and file sizes; and
- maximum line length.

The generated `STATIC_AUDIT.json` reports `PASS`. This is not a substitute for elaboration or kernel checking, but it catches packaging and obvious-source failures independently of Lean.

17.4 Expected build cost

The definition module adds no large inductive syntax, normalization semantics, or recursive proof over large-Veblen terms. The generic module has fewer than 200 lines, and the comparison module fewer than 90. Most declarations are small structural recursors or theorem applications. It is therefore plausible that the overlay fits the 15-second definition and 60-second proof budgets on the repository's reference class of machine. This is a prediction, not a measurement; only a clean Lean 4.34.0 build can certify it.

17.5 Repository integration

The proposed integration steps are:

1. copy the three `BusyLean/*.lean` files and the two `test/*.lean` files into the repository;
2. append the three imports supplied in `integration/BusyLean.lean.append`;
3. in `scripts/check.py`, set

```
PREVIOUS = "BusyLean.Comparison19"
CURRENT_DEF = "BusyLean.Contender20".
```

4. build `BusyLean.Contender20` and `BusyLean.Comparison20`;
5. run the smoke and focused audit files; and
6. after success, merge the focused audit declarations into the repository's global `test/Audit.lean`.

18 Verification boundary and risk register

A serious proposal should separate proved mathematics, source-level evidence, and unverified assumptions.

Claim	Current evidence	Remaining action
Definitions are constructive	All new recursors are structural; static audit finds no shortcut tokens	Confirm generated declaration kinds and axioms in Lean
The generic strictness proof is logically sound	Full proof given in Sections 5–8 and encoded in Lean	Elaborate and kernel-check on 4.34.0
The imports and names match the base repository	Source was written against commit `64d31a0...` and exact current interfaces	Build overlay against that commit
The proof stays within axiom policy	No new classical argument; audit commands included	Inspect `#print axioms` output
The closure avoids earlier contender constants	Definition uses `F19`, not `contender_19`; closure command included	Run environment closure audit
The time budgets are met	Modules are small and reuse compiled theory	Run clean budget script on reference hardware
The outer rank is qualitatively beyond finite rank	W-tree recursion and spliced limit nodes are explicit	Formalize finite-iterate simulation and separation theorems
A precise ordinal calibration is known	Not claimed	Develop semantics for hyperlift expressions

The largest practical risk is not the mathematics but elaboration detail: definitional reduction around `spliceLifts` or a changed implicit argument could require a local `change` or `simpa`. The proof was written in the style of the repository's current `HyperReflectionFacts.lean`, and all such equalities are shallow. No repair should alter the mathematical definitions.

19 Further research program

The proposal opens several directions that are more interesting than merely repeating it with a larger seed.

19.1 Immediate formal tasks

1. **Kernel build and timing.** Produce a clean Lean 4.34.0 receipt, axiom report, closure count, and definition/proof timings.

2. **Chain simulation.** Define a finite chain tree C_k and prove an explicit recurrence for the stage

$$\mathsf{Run}(\mathsf{Step}_S, L, C_k).$$

3. **Canonical limit tree.** Define $C_\omega = \mathsf{node}(k \mapsto C_k)$ and prove that its child coordinates expose every finite stage.

4. **Rank monotonicity without oracle monotonicity.** Identify an intensional embedding relation on lift expressions under which replacing a rank tree by a tree containing it preserves a strict lower bound.

19.2 Semantics and ordinal analysis

Assign formal syntax to transformer, lift, and hyperlift expressions, with an interpretation into the existing functions. A well-founded rank semantics could then prove lower bounds of the form

$$\text{expression rank } \alpha \quad \Longrightarrow \quad \text{dominates a calibrated hierarchy through } \alpha.$$

The nonmonotone oracle semantics means that the usual pointwise order is too coarse. A logical relation tracking explicit seed occurrences and selectable child coordinates is a more promising invariant.

19.3 Higher type levels

Contender 20 adds one fixed type level, $\mathsf{HLift} = \mathsf{Lift} \to \mathsf{Lift}$. A systematic successor would code a hierarchy

$$X_0 = \mathsf{Tr}, \qquad X_{k+1} = X_k \to X_k$$

and interpret generic splice/stage syntax at each finite level. The level could then depend on the input. This would parallel the earlier transition from fixed F_ω kinds to dynamic and laddered universes, but on the amplifier axis rather than the object-language axis.

19.4 Reflecting the amplifier language

A stronger and more difficult construction would give the object language codes for good transformer lifts or hyperlifts, normalize those codes, and reflect over every bounded program defining such an amplifier. This would replace the hand-selected `rankStep` by diagonalization over an explicit amplifier language. Soundness would require a typed syntax whose semantics preserves Good, making the invariant itself part of the type system.

19.5 Collapsing-function universe ladders

The language axis still has room above large Veblen. A canonical system of Buchholz-style or operator-controlled collapsing terms, with Lean-checked normal forms and well-foundedness, could feed `FL.wfUniv`. The main challenge is eliminating descending chains admitted by raw terms. Once such a ladder exists, it composes immediately with the W-rank amplifier developed here.

19.6 Minimality and compression

The generic module is intentionally explicit. After a successful build, one can investigate whether `GoodLift` should be a structure, whether the two strictness predicates admit a common typeclass, and whether the first- and second-order stage proofs can be generated from a single universe-polymorphic schema. Compression should follow validation, not precede it.

20 Conclusion

Contender 20 is designed as a principled next step rather than a cosmetic increase. Contender 19's language remains the base: programs range over a large-Veblen-length universe ladder with certified normalization. Its natural-number rank hierarchy remains as an enormous seed. The new operation raises the amplifier one type level and closes it under structural recursion on countably branching well-founded trees.

The core theorem is robust because it uses only explicit occurrences. At an outer node, child zero is below the parent by induction and second-order strict inflation; the splice makes that child definitionally visible. No oracle monotonicity, ordinal quotient, or semantic shortcut enters the formal comparison. This yields the exact pointwise inequality $\mathrm{family}19(n) < F_{20}(n)$, the closed comparison $\mathrm{contender}_{19} < \mathrm{contender}_{20}$, and a retained upper bound for every enumerated large-Veblen-ladder program.

The proposal's mathematical architecture is complete and the source package is ready for integration. Its one unresolved status item is mechanical but essential: the supplied Lean code must be built and audited on the repository's Lean 4.34.0 toolchain. The package makes that boundary visible rather than substituting confidence for a kernel receipt.

A Complete generic Lean module

```
import BusyLean.HyperReflectionFacts

/-!
# W-rank hyperreflection

`diagonalLift` is indexed by a natural-number rank.  This module raises the
construction one type level.  A `HyperLift` transforms transformer lifts, and
`runHyperLift` iterates such an operation along a countably branching
well-founded `WTree Nat`.

The important proof invariant is `GoodLift`: a lift preserves progressive
transformers and is strictly inflationary.  No monotonicity in an oracle is
used.  The strict comparison follows an explicit child and an explicit splice
coordinate, exactly as in the first-order reflection proofs.

All definitions are structural and executable.
-/
```

```
namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- One type level above `TransformerLift`. -/
abbrev HyperLift := TransformerLift → TransformerLift

/-- The two invariants needed to iterate a transformer lift safely. -/
abbrev GoodLift (lift : TransformerLift) : Prop :=
  PreservesProgressive lift ∧ StrictlyInflationary lift

/-- A hyperlift maps good lifts to good lifts. -/
abbrev PreservesGoodLift (op : HyperLift) : Prop :=
  ∀ lift, GoodLift lift → GoodLift (op lift)

/-- A hyperlift strictly exceeds its input lift at every visible coordinate. -/
abbrev StrictlyInflationary2 (op : HyperLift) : Prop :=
  ∀ (lift : TransformerLift), GoodLift lift →
    ∀ (base : Transformer), Progressive base →
      ∀ (seed : Family) (i o n : Nat), i ≤ n →
        lift base seed i n < op lift base seed o n

/-- Assemble a countable family of transformer lifts.  Output coordinate `i`
selects lift `i`; that lift is read at its canonical transformer coordinate
`0`, through `spliceTransformers`. -/
def spliceLifts (lifts : Nat → TransformerLift) : TransformerLift :=
  fun base => spliceTransformers (fun i => lifts i base)

/-- One rank step over a fixed schedule.  The input lift is used twice: first
to amplify the base and then as the local lift throughout the schedule. -/
def rankStep (schedule : WTree Nat) : HyperLift :=
  fun lift base => stageLift lift (lift base) schedule

/-- Iterate a hyperlift along a well-founded tree.

At a leaf, apply the hyperlift once.  At a node, recursively construct one lift
per child, splice those lifts, and apply the hyperlift to the splice. -/
def runHyperLift (op : HyperLift) (seed : TransformerLift) :
    WTree Nat → TransformerLift
  | .leaf => op seed
  | .node f =>
      op (spliceLifts (fun i => runHyperLift op seed (f i)))

/-- W-tree-indexed diagonal lifting over a fixed inner schedule. -/
def treeDiagonalLift (schedule rankTree : WTree Nat)
    (seed : TransformerLift) : TransformerLift :=
  runHyperLift (rankStep schedule) seed rankTree

/-- Splicing preserves the progressive-preservation invariant. -/
theorem spliceLifts_preservesProgressive {lifts : Nat → TransformerLift}
    (hlifts : ∀ i, PreservesProgressive (lifts i)) :
    PreservesProgressive (spliceLifts lifts) := by
  intro base hbase
  exact spliceTransformers_progressive (fun i => hlifts i base hbase)

/-- Splicing preserves strict inflationarity. -/
theorem spliceLifts_strictlyInflationary {lifts : Nat → TransformerLift}
    (hlifts : ∀ i, StrictlyInflationary (lifts i)) :
    StrictlyInflationary (spliceLifts lifts) := by
```

```
  intro base hbase seed i o n hi
  change base seed i n < lifts o base seed 0 n
  exact hlifts o base hbase seed i 0 n hi

/-- A splice of good lifts is good. -/
theorem spliceLifts_good {lifts : Nat → TransformerLift}
    (hlifts : ∀ i, GoodLift (lifts i)) :
    GoodLift (spliceLifts lifts) :=
  ⟨spliceLifts_preservesProgressive (fun i => (hlifts i).1),
    spliceLifts_strictlyInflationary (fun i => (hlifts i).2)⟩

/-- One rank step maps good lifts to good lifts. -/
theorem rankStep_preservesGood (schedule : WTree Nat) :
    PreservesGoodLift (rankStep schedule) := by
  intro lift hlift
  rcases hlift with ⟨hpres, hstrict⟩
  constructor
  · intro base hbase
    change Progressive (stageLift lift (lift base) schedule)
    exact stageLift_progressive lift hpres
      (lift base) (hpres base hbase) schedule
  · intro base hbase seed i o n hi
    change base seed i n <
      stageLift lift (lift base) schedule seed o n
    have hfirst : base seed i n < lift base seed i n :=
      hstrict base hbase seed i i n hi
    have hsecond : lift base seed i n <
        stageLift lift (lift base) schedule seed o n :=
      base_lt_stageLift lift hpres hstrict
        (lift base) (hpres base hbase) schedule seed i o n hi
    exact Nat.lt_trans hfirst hsecond

/-- One rank step is strictly inflationary at the level of lifts. -/
theorem rankStep_strictlyInflationary2 (schedule : WTree Nat) :
    StrictlyInflationary2 (rankStep schedule) := by
  intro lift hlift base hbase seed i o n hi
  rcases hlift with ⟨hpres, hstrict⟩
  change lift base seed i n <
    stageLift lift (lift base) schedule seed o n
  exact base_lt_stageLift lift hpres hstrict
    (lift base) (hpres base hbase) schedule seed i o n hi

/-- W-tree iteration preserves goodness. -/
theorem runHyperLift_good (op : HyperLift)
    (hop : PreservesGoodLift op)
    (seed : TransformerLift) (hseed : GoodLift seed) :
    (a : WTree Nat) → GoodLift (runHyperLift op seed a)
  | .leaf => hop seed hseed
  | .node f =>
      hop
        (spliceLifts (fun i => runHyperLift op seed (f i)))
        (spliceLifts_good
          (fun i => runHyperLift_good op hop seed hseed (f i)))

/-- Every W-tree stage strictly exceeds its seed lift.

The node case follows child zero, identifies it with coordinate zero of the
splice, and then applies strictness of the hyperlift. -/
theorem seed_lt_runHyperLift (op : HyperLift)
    (hpres : PreservesGoodLift op)
```

```
    (hstrict : StrictlyInflationary2 op)
    (seedLift : TransformerLift) (hseed : GoodLift seedLift) :
    (a : WTree Nat) →
      ∀ (base : Transformer), Progressive base →
        ∀ (seed : Family) (i o n : Nat), i ≤ n →
          seedLift base seed i n <
            runHyperLift op seedLift a base seed o n
  | .leaf, base, hbase, seed, i, o, n, hi =>
      hstrict seedLift hseed base hbase seed i o n hi
  | .node f, base, hbase, seed, i, o, n, hi => by
      have hchild :
          seedLift base seed i n <
            runHyperLift op seedLift (f 0) base seed 0 n :=
        seed_lt_runHyperLift op hpres hstrict seedLift hseed
          (f 0) base hbase seed i 0 n hi
      have hsplice : GoodLift
          (spliceLifts (fun j => runHyperLift op seedLift (f j))) :=
        spliceLifts_good
          (fun j => runHyperLift_good op hpres seedLift hseed (f j))
      have hparent := hstrict
        (spliceLifts (fun j => runHyperLift op seedLift (f j)))
        hsplice base hbase seed 0 o n (Nat.zero_le n)
      change
        runHyperLift op seedLift (f 0) base seed 0 n <
          runHyperLift op seedLift (.node f) base seed o n at hparent
      exact Nat.lt_trans hchild hparent

/-- The W-rank diagonal lift is good whenever its seed lift is good. -/
theorem treeDiagonalLift_good (schedule rankTree : WTree Nat)
    (seed : TransformerLift) (hseed : GoodLift seed) :
    GoodLift (treeDiagonalLift schedule rankTree seed) := by
  simpa [treeDiagonalLift] using
    runHyperLift_good (rankStep schedule)
      (rankStep_preservesGood schedule) seed hseed rankTree

/-- The W-rank diagonal lift strictly exceeds its seed lift. -/
theorem seed_lt_treeDiagonalLift (schedule rankTree : WTree Nat)
    (seedLift : TransformerLift) (hseed : GoodLift seedLift)
    (base : Transformer) (hbase : Progressive base)
    (seed : Family) (i o n : Nat) (hi : i ≤ n) :
    seedLift base seed i n <
      treeDiagonalLift schedule rankTree seedLift base seed o n := by
  simpa [treeDiagonalLift] using
    seed_lt_runHyperLift (rankStep schedule)
      (rankStep_preservesGood schedule)
      (rankStep_strictlyInflationary2 schedule)
      seedLift hseed rankTree base hbase seed i o n hi

end BusyLean
```

B Complete contender definition

```
import BusyLean.TransfiniteHyperReflection
import BusyLean.Contender19

/-!
# Contender 20: W-rank hyperreflection
```

```
```
family19 n   := F19 (F19 n)
seedLift20 n   := diagonalLift (schedule20 n) (family19 n + 1)
lift20 n   := treeDiagonalLift (schedule20 n) (rankTree20 n) (seedLift20 n)
F20 n   := lift20 n lTransformer seedFamily20 n n
contender_20   := F20 (F20 42)
```

Contender 19 uses a natural-number rank of transformer lifts.  Contender 20
raises the rank mechanism one type level and iterates it along a countably
branching well-founded tree.  At an outer node, the recursively generated child
lifts are spliced into a single lift, then one diagonal rank step is applied.

The seed is already a diagonal lift of rank `family19 n + 1`.  The outer rank
tree is an autonomous hyperclosure above a schedule that is itself an
autonomous hyperclosure above Contender 19's schedule.  The object-language
transformer remains the large-Veblen-ladder reflector.

Do not evaluate `contender_20`.
-/

namespace BusyLean

open BigNumber (WTree)
open BusyLean.W

/-- Contender 19's construction as a function of its input. -/
def family19 (n : Nat) : Nat := F19 (F19 n)

/-- The constant oracle family used by Contender 20. -/
def seedFamily20 : Family := fun _ => family19

/-- Inner schedule: a self-height autonomous closure above Contender 19's
schedule. -/
def schedule20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42) (Autonomous.encode (schedule19 n)))

/-- Outer W-rank: another self-height autonomous closure, now above the inner
schedule itself. -/
def rankTree20 (n : Nat) : WTree Nat :=
  Autonomous.realize
    (Autonomous.hyper (family19 n + 42) (Autonomous.encode (schedule20 n)))

/-- The finite-rank lift from which the W-rank closure starts. -/
def seedLift20 (n : Nat) : TransformerLift :=
  diagonalLift (schedule20 n) (family19 n + 1)

/-- The W-rank lift used at input `n`. -/
def lift20 (n : Nat) : TransformerLift :=
  treeDiagonalLift (schedule20 n) (rankTree20 n) (seedLift20 n)

/-- Pointwise Contender-20 family. -/
def F20 (n : Nat) : Nat :=
  lift20 n lTransformer seedFamily20 n n

/-- Contender 20.  Never evaluate it. -/
def contender_20 : Nat := F20 (F20 42)
```

```
end BusyLean
```

C Complete comparison module

```
import BusyLean.Contender20
import BusyLean.Comparison19

/-! # Contender 20: comparison theorems -/

namespace BusyLean

open BusyLean.W

/-- The finite-rank seed lift satisfies both iteration invariants. -/
theorem seedLift20_good (n : Nat) : GoodLift (seedLift20 n) :=
  ⟨diagonalLift_preservesProgressive (schedule20 n) (family19 n + 1),
    diagonalLift_strictlyInflationary (schedule20 n) (family19 n + 1)⟩

/-- The full W-rank lift satisfies both iteration invariants. -/
theorem lift20_good (n : Nat) : GoodLift (lift20 n) := by
  simpa [lift20] using
    treeDiagonalLift_good (schedule20 n) (rankTree20 n)
      (seedLift20 n) (seedLift20_good n)

/-- The raw large-Veblen reflection exceeds the Contender-19 seed family. -/
theorem family19_lt_raw20 (n : Nat) :
    family19 n < lTransformer seedFamily20 0 n := by
  change seedFamily20 0 n < lTransformer seedFamily20 0 n
  exact lTransformer_progressive seedFamily20 0 0 n (Nat.zero_le n)

/-- The finite diagonal rank strictly exceeds raw reflection. -/
theorem raw20_lt_seedLift20 (n : Nat) :
    lTransformer seedFamily20 0 n <
      seedLift20 n lTransformer seedFamily20 0 n := by
  simpa [seedLift20] using
    (diagonalLift_strictlyInflationary
      (schedule20 n) (family19 n + 1))
      lTransformer lTransformer_progressive
      seedFamily20 0 0 n (Nat.zero_le n)

/-- The outer W-rank stage strictly exceeds its already enormous seed lift. -/
theorem seedLift20_lt_F20 (n : Nat) :
    seedLift20 n lTransformer seedFamily20 0 n < F20 n := by
  change seedLift20 n lTransformer seedFamily20 0 n <
    lift20 n lTransformer seedFamily20 n n
  simpa [lift20] using
    seed_lt_treeDiagonalLift (schedule20 n) (rankTree20 n)
      (seedLift20 n) (seedLift20_good n)
      lTransformer lTransformer_progressive
      seedFamily20 0 n n (Nat.zero_le n)

/-- Raw reflection is below the complete Contender-20 stage. -/
theorem raw_reflection20_lt_F20 (n : Nat) :
    reflectLFamily seedFamily20 n < F20 n := by
  change lTransformer seedFamily20 0 n < F20 n
  exact Nat.lt_trans (raw20_lt_seedLift20 n) (seedLift20_lt_F20 n)
```

```
/-- Pointwise domination of Contender 19's family. -/
theorem family19_lt_F20 (n : Nat) : family19 n < F20 n :=
  Nat.lt_trans (family19_lt_raw20 n) (raw_reflection20_lt_F20 n)

/-- Every large-Veblen-ladder program in the bounded enumeration lies below
`F20 n`. -/
theorem lvalue_lt_F20 {n : Nat} {t : FL.CTm Ordinal.LargeVeblen}
    (ht : t ∈ FL.lveblenLadderUpTo n) :
    FL.lvalue (packFamily seedFamily20) n t < F20 n :=
  Nat.lt_trans (lvalue_lt_reflectLFamily seedFamily20 ht)
    (raw_reflection20_lt_F20 n)

/-- The input is below Contender 19's self-composed family. -/
theorem input_lt_family19 (n : Nat) : n < family19 n :=
  Nat.lt_trans (input_lt_F19 n) (input_lt_F19 (F19 n))

/-- Every input is below the corresponding Contender-20 family value. -/
theorem input_lt_F20 (n : Nat) : n < F20 n :=
  Nat.lt_trans (input_lt_family19 n) (family19_lt_F20 n)

/-- The required formal comparison. -/
theorem contender_19_lt_contender_20 : contender_19 < contender_20 :=
  Nat.lt_trans (family19_lt_F20 42) (input_lt_F20 (F20 42))

/-- A convenient inherited comparison. -/
theorem contender_18_lt_contender_20 : contender_18 < contender_20 :=
  Nat.lt_trans contender_18_lt_contender_19 contender_19_lt_contender_20

end BusyLean
```

D Reproducibility checklist

1. Check out BusyLean commit `64d31a0cf6f1aa4477a088be5554957de6e87a61`.
2. Confirm `lean-toolchain` contains `leanprover/lean4:v4.34.0`.
3. Copy the overlay files into the checkout.
4. Append the imports in `integration/BusyLean.lean.append`.
5. Run `python3 scripts/static_audit.py`.
6. Run `lake build BusyLean.Contender20`.
7. Run `lake build BusyLean.Comparison20`.
8. Run `lake env lean test/Contender20Smoke.lean`.
9. Run `lake env lean test/Audit20.lean`.
10. Update and run the repository-wide clean budget script.
11. Record build times, closure size, and exact axiom lists in a receipt.

References

[1] Scott Aaronson, *Who Can Name the Bigger Number?*, https://www.scottaaronson.com/writings/bignumbers.html.

[2] Cody Roux, *name-the-biggest-number*, Coq contest repository, https://github.com/codyroux/name-the-biggest-number.

[3] Vladimir Reshetnikov, *BusyLean: name the biggest number, in Lean*, https://github.com/VladimirReshetnikov/BusyLean, base commit `64d31a0cf6f1aa4477a088be5554957de6e87a61`.

[4] Leonardo de Moura and Sebastian Ullrich, "The Lean 4 Theorem Prover and Programming Language," in *Automated Deduction–CADE 28*, Lecture Notes in Computer Science 12699, Springer, 2021, pp. 625–635.

[5] Kurt Schütte, *Proof Theory*, Grundlehren der mathematischen Wissenschaften 225, Springer, 1977.

[6] Wolfram Pohlers, *Proof Theory: The First Step into Impredicativity*, Universitext, Springer, 2009.

[7] Helmut Schwichtenberg and Stanley S. Wainer, *Proofs and Computations*, Perspectives in Logic, Cambridge University Press, 2012.